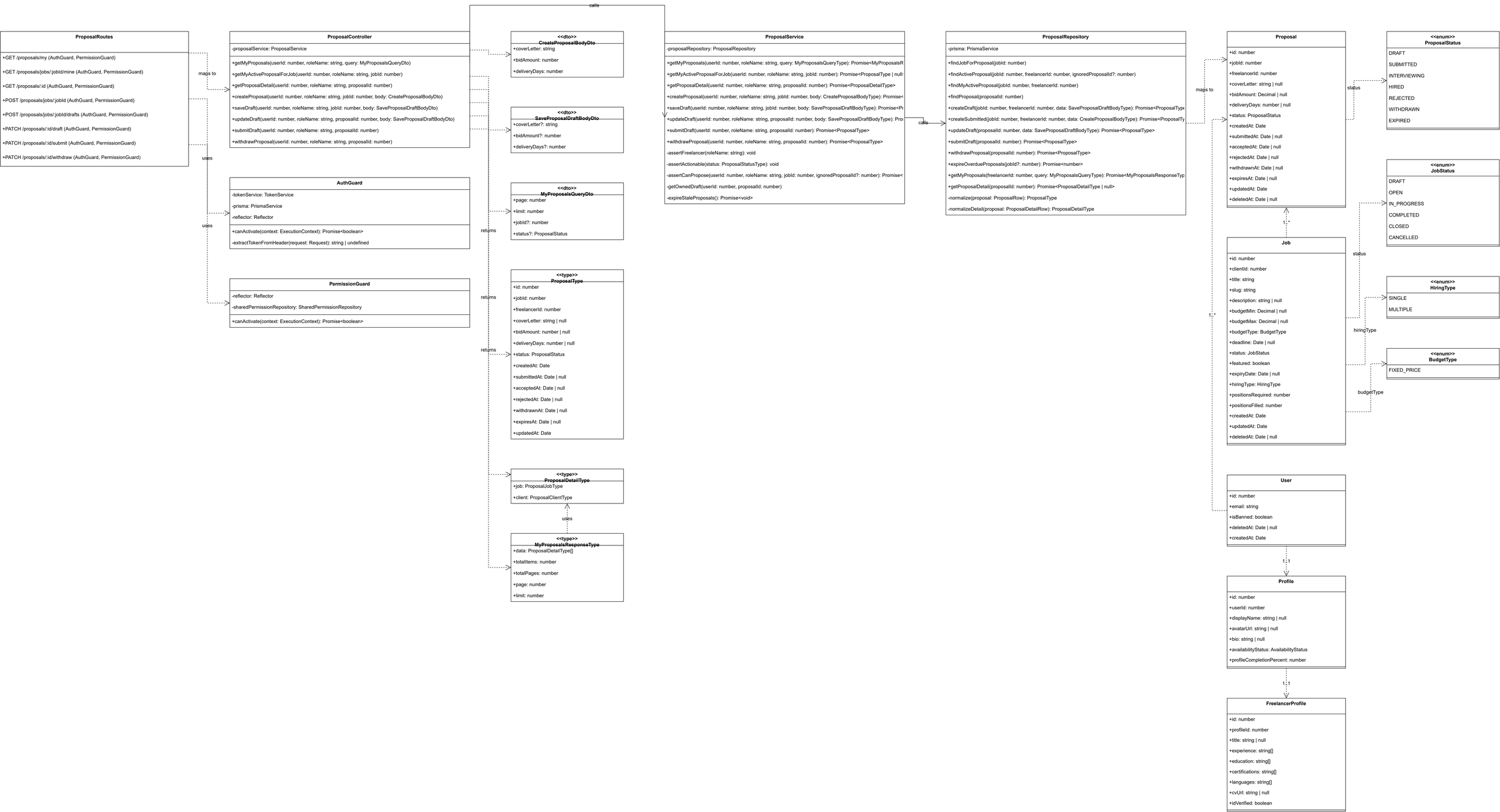

ProposalRoutes
+GET /proposals/my (AuthGuard, PermissionGuard)
+GET /proposals/jobs/:jobId/mine (AuthGuard, PermissionGuard)
+GET /proposals/:id (AuthGuard, PermissionGuard)
+POST /proposals/jobs/:jobId (AuthGuard, PermissionGuard)
+POST /proposals/jobs/:jobId/drafts (AuthGuard, PermissionGuard)
+PATCH /proposals/:id/draft (AuthGuard, PermissionGuard)
+PATCH /proposals/:id/submit (AuthGuard, PermissionGuard)
+PATCH /proposals/:id/withdraw (AuthGuard, PermissionGuard)
maps to
uses
uses
ProposalController
-proposalService: ProposalService
+getMyProposals(userId: number, roleName: string, query: MyProposalsQueryDto)
+getMyActiveProposalForJob(userId: number, roleName: string, jobId: number)
+getProposalDetail(userId: number, roleName: string, proposalId: number)
+createProposal(userId: number, roleName: string, jobId: number, body: CreateProposalBodyDto)
+saveDraft(userId: number, roleName: string, jobId: number, body: SaveProposalDraftBodyDto)
+updateDraft(userId: number, roleName: string, proposalId: number, body: SaveProposalDraftBodyDto)
+submitDraft(userId: number, roleName: string, proposalId: number)
+withdrawProposal(userId: number, roleName: string, proposalId: number)
AuthGuard
-tokenService: TokenService
-prisma: PrismaService
-reflector: Reflector
+canActivate(context: ExecutionContext): Promise<boolean>
-extractTokenFromHeader(request: Request): string | undefined
PermissionGuard
-reflector: Reflector
-sharedPermissionRepository: SharedPermissionRepository
+canActivate(context: ExecutionContext): Promise<boolean>
calls
returns
returns
returns
<<dto>>
CreateProposalBodyDto
+coverLetter: string
+bidAmount: number
+deliveryDays: number
<<dto>>
SaveProposalDraftBodyDto
+coverLetter?: string
+bidAmount?: number
+deliveryDays?: number
<<dto>>
MyProposalsQueryDto
+page: number
+limit: number
+jobId?: number
+status?: ProposalStatus
<<type>>
ProposalType
+id: number
+jobId: number
+freelancerId: number
+coverLetter: string | null
+bidAmount: number | null
+deliveryDays: number | null
+status: ProposalStatus
+createdAt: Date
+submittedAt: Date | null
+acceptedAt: Date | null
+rejectedAt: Date | null
+withdrawnAt: Date | null
+expiresAt: Date | null
+updatedAt: Date
<<type>>
ProposalDetailType
+job: ProposalJobType
+client: ProposalClientType
uses
<<type>>
MyProposalsResponseType
+data: ProposalDetailType[]
+totalItems: number
+totalPages: number
+page: number
+limit: number
ProposalService
-proposalRepository: ProposalRepository
+getMyProposals(userId: number, roleName: string, query: MyProposalsQueryType): Promise<MyProposalsR
+getMyActiveProposalForJob(userId: number, roleName: string, jobId: number): Promise<ProposalType | null>
+getProposalDetail(userId: number, roleName: string, proposalId: number): Promise<ProposalDetailType>
+createProposal(userId: number, roleName: string, jobId: number, body: CreateProposalBodyType): Promise<
+saveDraft(userId: number, roleName: string, jobId: number, body: SaveProposalDraftBodyType): Promise<Pr
+updateDraft(userId: number, roleName: string, proposalId: number, body: SaveProposalDraftBodyType): Pro
+submitDraft(userId: number, roleName: string, proposalId: number): Promise<ProposalType>
+withdrawProposal(userId: number, roleName: string, proposalId: number): Promise<ProposalType>
-assertFreelancer(roleName: string): void
-assertActionable(status: ProposalStatusType): void
-assertCanPropose(userId: number, roleName: string, jobId: number, ignoredProposalId?: number): Promise<
-getOwnedDraft(userId: number, proposalId: number)
-expireStaleProposals(): Promise<void>
calls
ProposalRepository
-prisma: PrismaService
+findJobForProposal(jobId: number)
+findActiveProposal(jobId: number, freelancerId: number, ignoredProposalId?: number)
+findMyActiveProposal(jobId: number, freelancerId: number)
+findProposal(proposalId: number)
+createDraft(jobId: number, freelancerId: number, data: SaveProposalDraftBodyType): Promise<ProposalType
+createSubmitted(jobId: number, freelancerId: number, data: CreateProposalBodyType): Promise<ProposalTy
+updateDraft(proposalId: number, data: SaveProposalDraftBodyType): Promise<ProposalType>
+submitDraft(proposalId: number): Promise<ProposalType>
+withdrawProposal(proposalId: number): Promise<ProposalType>
+expireOverdueProposals(jobId?: number): Promise<number>
+getMyProposals(freelancerId: number, query: MyProposalsQueryType): Promise<MyProposalsResponseTyp
+getProposalDetail(proposalId: number): Promise<ProposalDetailType | null>
-normalize(proposal: ProposalRow): ProposalType
-normalizeDetail(proposal: ProposalDetailRow): ProposalDetailType
maps to
Proposal
+id: number
+jobId: number
+freelancerId: number
+coverLetter: string | null
+bidAmount: Decimal | null
+deliveryDays: number | null
+status: ProposalStatus
+createdAt: Date
+submittedAt: Date | null
+acceptedAt: Date | null
+rejectedAt: Date | null
+withdrawnAt: Date | null
+expiresAt: Date | null
+updatedAt: Date
+deletedAt: Date | null
1..*
Job
+id: number
+clientId: number
+title: string
+slug: string
+description: string | null
+budgetMin: Decimal | null
+budgetMax: Decimal | null
+budgetType: BudgetType
+deadline: Date | null
+status: JobStatus
+featured: boolean
+expiryDate: Date | null
+hiringType: HiringType
+positionsRequired: number
+positionsFilled: number
+createdAt: Date
+updatedAt: Date
+deletedAt: Date | null
1..*
User
+id: number
+email: string
+isBanned: boolean
+deletedAt: Date | null
+createdAt: Date
1..1
Profile
+id: number
+userId: number
+displayName: string | null
+avatarUrl: string | null
+bio: string | null
+availabilityStatus: AvailabilityStatus
+profileCompletionPercent: number
1..1
FreelancerProfile
+id: number
+profileId: number
+title: string | null
+experience: string[]
+education: string[]
+certifications: string[]
+languages: string[]
+cvUrl: string | null
+idVerified: boolean
status
status
hiringType
budgetType
<<enum>>
ProposalStatus
DRAFT
SUBMITTED
INTERVIEWING
HIRED
REJECTED
WITHDRAWN
EXPIRED
<<enum>>
JobStatus
DRAFT
OPEN
IN_PROGRESS
COMPLETED
CLOSED
CANCELLED
<<enum>>
HiringType
SINGLE
MULTIPLE
<<enum>>
BudgetType
FIXED_PRICE